BFP/1: a Binary Fetch Protocol

Bharath | Specification, version 1

Status: version 1. Words in capitals (MUST, SHOULD, MAY) have their usual RFC 2119 meaning.

1. Model

BFP runs over one TCP connection. The client sends requests and the server answers each one. A connection carries any number of exchanges, one after another: the client MUST NOT send a request until the previous response has ended, and the server never closes a connection because an exchange finished or failed. Either side may close it at any other time (a client does so when it is done, or on a protocol failure). All multi-byte integers are unsigned and big-endian. Everything on the wire is a sequence of frames.

2. Frame header

Every frame is an 8-byte header followed by `Length` bytes of payload.

Offset	Field	Size	Meaning
0	Length	24 bits	Payload size in bytes, header not included
3	Type	8 bits	Frame type (section 3)
4	Flags	8 bits	Bit flags, meaning depends on Type. Senders MUST set unused bits to 0, receivers MUST ignore them.
5	Stream	24 bits	Identifies one exchange. A response carries the stream id of its request. 0 is reserved. Clients SHOULD use odd ids (1, 3, 5, ...).

A sender MUST NOT send a payload longer than 16384 bytes (2^14). A receiver discards the payload of such a frame. If it is a REQUEST the server answers 400; if it is a RESPONSE or DATA frame on the client's own stream, the response has failed (section 6).

Why these widths. *Length, 24 bits:* v1 only needs 14 bits (16384), but a 16-bit field would cap every future version at 64 KiB per frame. 24 bits is the smallest byte-aligned field that lets a v2 raise the limit to 16 MiB without changing the header layout, and it costs one byte per frame (0.006% at full size). *Type, 8 bits:* v1 uses three of 256 values, which leaves room, and one byte keeps the skip rule (section 3) trivial to implement. *Flags, 8 bits:* v1 defines one flag (END_STREAM, 0x01), and the other seven are growth space. *Stream, 24 bits:* v1 is strictly sequential, so the id is only a label today. It exists so that v2 can interleave responses without a new header. 16M exchanges per connection is plenty, and a connection that runs out is simply closed and reopened. 24 bits also keeps the header at exactly 8 bytes, one hexdump row. HTTP/2 chose 24/8/8/1+31 (9 bytes): the same length reasoning (default 16384, ceiling 16 MiB), 31 stream bits because ids are never reused on a long-lived multiplexed connection, and one reserved bit for the future. BFP v1 is sequential and has no use for either, so it spends neither.

3. Frame types

Type	Name	Payload	Flags
0x01	REQUEST	`method` u8, `path_len` u16, `path` (UTF-8, `path_len` bytes), header block (rest of payload)	END_STREAM 0x01, MUST be set (v1 requests have no body)
0x02	RESPONSE	`status` u16, header block (rest of payload)	END_STREAM 0x01, set if no DATA frame follows
0x03	DATA	raw body bytes	END_STREAM 0x01 on the last DATA frame of the response

`method` is 0x01 (GET), the only method in v1. The path starts with `/`. A response is one RESPONSE frame followed by zero or more DATA frames on the same stream, each at most 16384 bytes. The last frame of the response carries END_STREAM.

Unknown frames MUST be skipped cleanly. A receiver that gets a frame whose Type it does not know (or one that makes no sense in that direction, such as DATA arriving at a server) reads exactly `Length` payload bytes, discards them, sends nothing back, and carries on with the next frame. It MUST NOT close the connection or report an error, and it ignores the frame's flags (END_STREAM included). This rule holds in both directions. It is how v2 adds frame types without breaking v1 peers.

4. Header block

A header block is a sequence of entries filling the rest of the payload. It may be empty. Each entry is one of two forms, chosen by its first byte.

- **Indexed name** (`1iiiiiii`, byte `0x80 | i`, i in 1..10): the name comes from the static table below. The byte is followed by `value_len` u8 and `value` bytes.
- **Literal name** (`0x00`): followed by `name_len` u8 (at least 1), the name, `value_len` u8, then the value. Names are lowercase ASCII.

Any other first byte (`0x01-0x7F`, `0x8B-0xFF`) makes the whole block malformed. Values are at most 255 bytes. A name may repeat (receivers use the first occurrence), and any header may appear in either direction: receivers ignore the ones they do not need.

i	Name	Normally sent by	Value
1	content-type	server	media type, e.g. `text/html`
2	content-length	server	body size, decimal ASCII
3	host	client	`host:port` as typed
4	user-agent	client	e.g. `bcurl/1`
5	accept	client	`*/*`
6	server	server	`bserve/1`
7	connection	client	`keep-alive`
8	last-modified	server	file mtime, decimal Unix seconds
9	etag	server	`<size hex>-<mtime hex>`
10	cache-control	server	`no-cache`

A header that is not in the table (for example `x-trace: 42`) is sent as a literal. These two forms are the first two ideas of HPACK, a static table and length-prefixed literals; there is no dynamic table and no Huffman coding. Indexing `content-type` costs 1 byte instead of 14.

5. Server behaviour

1. Read one frame. Skip it if the type is not REQUEST (section 3).
2. The request is **malformed**, and the server answers **400**, if: Length exceeds 16384; END_STREAM is not set; the stream id is 0; `method` is not 0x01; `path_len` runs past the payload; the path is not valid UTF-8, contains a NUL byte, or does not start with `/`; or the header block is malformed.
3. Otherwise map the path to a file. A path ending in `/` has `index.html` appended. Join it to the root, resolve `.`, `..` and symlinks, and if the result is not a regular file inside the root the answer is **404**. If the file exists but cannot be read the answer is **500**.
4. Reply `RESPONSE` (200, headers `content-type`, `content-length`, `server`, `last-modified`, `etag`, `cache-control`) then DATA frames. An error reply is a RESPONSE with `content-type: text/plain`, `content-length` and `server`, then one short DATA frame (`bad request`, `not found`, `internal error`).
5. Every reply uses the stream id of the request it answers. **After any reply, including an error, keep the connection open** and go back to step 1. If the peer closes in the middle of a frame, close quietly.

6. Client behaviour

Open one TCP connection, send REQUEST (stream 1) with headers `host`, `user-agent`, `accept`, `connection: keep-alive`, then read frames until a frame on that stream carries END_STREAM, skipping unknown frames and frames for other streams. Further requests to the same host reuse the connection with stream ids 3, 5, and so on. The client never opens a second connection. If `content-length` is present and the DATA bytes do not add up to it, the response has failed. A status of 400 or above is an error for the caller. A malformed header block, a frame out of order (DATA before RESPONSE, a second RESPONSE), an oversize frame on its stream, or the connection closing before END_STREAM all mean the response has failed: the client reports an error and closes the connection.

7. Versioning

There is no version number on the wire. A v2 adds new frame types and flags, never changes the meaning of v1 ones, and starts by sending a new frame (say SETTINGS, type 0x04) that a v1 peer skips silently under section 3. A v2 peer that gets no SETTINGS back simply stays in v1 mode. Static-table indices 11-127 are reserved for such extensions.

Annotated hexdump: one complete request and response

Bharath | captured from a real run, annotation generated from the captured bytes

Captured with `./bserve ./www 9000` and `./bcurl -v -H x-trace:42 localhost:9000/index.html`. The client printed 'hello, world' and exited 0. `index.html` holds the 13 bytes `hello, world\n`. The extra `-H` forces one literal header so that both header encodings appear. The request is one frame (76 bytes). The response is two frames, RESPONSE (69 bytes) and DATA (21 bytes). All three are annotated below, and the raw dump follows the tables.

1. Request frame (client to server)

Offset	Bytes (hex)	Field	Meaning
0000	00 00 44	Length	68: payload bytes after the 8-byte header
0003	01	Type	0x01 = REQUEST
0004	01	Flags	0x01 = END_STREAM
0005	00 00 01	Stream	1
0008	01	method	0x01 = GET
0009	00 0b	path_len	11
000b	2f 69 6e 64 65 78 2e 68 74 6d 6c	path	"/index.html"
0016	83 0e	indexed name	0x83 = 0x80 | 3, table entry 3 is host; value_len 14
0018	6c 6f 63 61 6c 68 6f 73 74 3a 39 30 30 30	value	"localhost:9000"
0026	84 07	indexed name	0x84 = 0x80 | 4, table entry 4 is user-agent; value_len 7
0028	62 63 75 72 6c 2f 31	value	"bcurl/1"
002f	85 03	indexed name	0x85 = 0x80 | 5, table entry 5 is accept; value_len 3
0031	2a 2f 2a	value	"*/*"
0034	87 0a	indexed name	0x87 = 0x80 | 7, table entry 7 is connection; value_len 10
0036	6b 65 65 70 2d 61 6c 69 76 65	value	"keep-alive"
0040	00 07	literal name	0x00 = literal; name_len 7
0042	78 2d 74 72 61 63 65	name	"x-trace"
0049	02	value_len	2
004a	34 32	value	"42"

The 8-byte header is followed by 68 payload bytes: three request fields, then four indexed headers (one byte of name each, from the static table) and one literal header. The literal name *x-trace* costs 9 bytes where an indexed name costs 1.

2. Response frame (server to client)

Offset	Bytes (hex)	Field	Meaning
0000	00 00 3d	Length	61: payload bytes after the 8-byte header
0003	02	Type	0x02 = RESPONSE
0004	00	Flags	0x00 = none
0005	00 00 01	Stream	1
0008	00 c8	status	200
000a	81 09	indexed name	0x81 = 0x80 | 1, table entry 1 is content-type; value_len 9
000c	74 65 78 74 2f 68 74 6d 6c	value	"text/html"
0015	82 02	indexed name	0x82 = 0x80 | 2, table entry 2 is content-length; value_len 2
0017	31 33	value	"13"
0019	86 08	indexed name	0x86 = 0x80 | 6, table entry 6 is server; value_len 8
001b	62 73 65 72 76 65 2f 31	value	"bserve/1"
0023	88 0a	indexed name	0x88 = 0x80 | 8, table entry 8 is last-modified; value_len 10
0025	31 37 39 31 34 36 30 38 30 30	value	"1791460800"
002f	89 0a	indexed name	0x89 = 0x80 | 9, table entry 9 is etag; value_len 10
0031	64 2d 36 61 63 37 38 35 63 30	value	"d-6ac785c0"
003b	8a 08	indexed name	0x8a = 0x80 | 10, table entry 10 is cache-control; value_len 8
003d	6e 6f 2d 63 61 63 68 65	value	"no-cache"

No END_STREAM here, because a DATA frame follows. Status 200 is the u16 `00 c8`. The server sent six headers, all indexed. *content-length* is the ASCII digits '13', and *last-modified* is Unix seconds, also as ASCII.

3. Data frame (server to client)

Offset	Bytes (hex)	Field	Meaning
0000	00 00 0d	Length	13: payload bytes after the 8-byte header
0003	03	Type	0x03 = DATA
0004	01	Flags	0x01 = END_STREAM
0005	00 00 01	Stream	1
0008	68 65 6c 6c 6f 2c 20 77 6f 72 6c 64 0a	body	"hello, world\n" (13 bytes)

The last frame of the response, so END_STREAM is set. The body is exactly the 13 bytes of the file, and the stream id (1) matches the request.

Raw dump as printed by `bcurl -v` (> sent, < received)

```
> REQUEST length=68 flags=0x01 stream=1
> 00000000  00 00 44 01 01 00 00 01  01 00 0b 2f 69 6e 64 65  |..D......../inde|
> 00000010  78 2e 68 74 6d 6c 83 0e  6c 6f 63 61 6c 68 6f 73  |x.html..localhos|
> 00000020  74 3a 39 30 30 30 84 07  62 63 75 72 6c 2f 31 85  |t:9000..bcurl/1.|
> 00000030  03 2a 2f 2a 87 0a 6b 65  65 70 2d 61 6c 69 76 65  |.*/*..keep-alive|
> 00000040  00 07 78 2d 74 72 61 63  65 02 34 32              |..x-trace.42|
> 0000004c
< RESPONSE length=61 flags=0x00 stream=1
< 00000000  00 00 3d 02 00 00 00 01  00 c8 81 09 74 65 78 74  |..=.........text|
< 00000010  2f 68 74 6d 6c 82 02 31  33 86 08 62 73 65 72 76  |/html..13..bserv|
< 00000020  65 2f 31 88 0a 31 37 39  31 34 36 30 38 30 30 89  |e/1..1791460800.|
< 00000030  0a 64 2d 36 61 63 37 38  35 63 30 8a 08 6e 6f 2d  |.d-6ac785c0..no-|
< 00000040  63 61 63 68 65                                    |cache|
< 00000045
< DATA length=13 flags=0x01 stream=1
< 00000000  00 00 0d 03 01 00 00 01  68 65 6c 6c 6f 2c 20 77  |........hello, w|
< 00000010  6f 72 6c 64 0a                                    |orld.|
< 00000015
```